

OCSF, Semantic Layer, and Central Catalogue

Purpose, with/without view, and concrete cyber examples

Executive summary

OCSF, semantic layer, and central catalogue solve different parts of the cyber data problem. OCSF standardizes security records. The semantic layer standardizes enterprise cyber meaning. The central catalogue makes data products, definitions, owners, lineage, quality, and access rules discoverable and governed.

Raw tools/logs -> OCSF normalized records -> Curated cyber entities -> Semantic layer -> Central catalogue -> Dashboards / Agents / NLQ / Governance

Capability	Purpose	Concrete example
OCSF	Common cyber record language.	Qualys, Tenable, Wiz findings all become normalized vulnerability findings with consistent severity, status, resource, and finding fields.
Semantic layer	Common cyber/business meaning.	"Critical app risk" means app has active critical vulns, internet-facing assets, privileged access, and SLA breach based on approved definitions.
Central catalogue	Discovery, trust, ownership, lineage, and policy metadata.	Agent finds certified asset_inventory_gold and vuln_findings_gold, sees owner, freshness, data quality, lineage, and access rules.

1. Why OCSF is needed

Security tools describe the same thing using different fields and event categories. OCSF reduces this by mapping vendor-specific records into a common schema. It is especially valuable in the silver layer of a data lake, where raw data has been cleaned and normalized but before enterprise-specific business meaning is added.

Problem without OCSF	OCSF value	Concrete example
Same concept has different field names.	Common field names and objects.	Tenable host_name, Qualys dns, Wiz resource.id, CrowdStrike aid are mapped to resource / device / endpoint concepts.
Severity is inconsistent.	Consistent severity model.	Qualys severity 5, Tenable Critical, Wiz Critical can normalize to severity=Critical / severity_id=5.
Each detection is vendor-specific.	Reusable detection logic.	Failed login, suspicious process, cloud API activity, and vulnerability finding queries can use common OCSF classes.
New source onboarding breaks consumers.	Downstream consumers depend on normalized records.	Adding a new scanner only needs source-to-OCSF mapping; the risk dashboard does not need full rewrite.

Concrete OCSF mapping examples

Use case	Raw source fields	OCSF-normalized concept
Vulnerability finding	Tenable: plugin_id, severity, dns_name, first_seen, last_seen	class=Vulnerability Finding; vulnerability.uid/plugin; severity; resource.name; finding.first_seen/last_seen; status.
Cloud exposure	AWS: instanceId, publicIpAddress, securityGroupId, accountId	resource.uid; cloud.account.uid; endpoint.ip; network/security group object; exposure context.
Authentication event	Okta/Azure AD: userPrincipalName, clientIP,	class=Authentication; actor.user.name;

	result, app, timestamp	src_endpoint.ip; status; service/app; time.
Endpoint activity	CrowdStrike: aid, event_simpleName, ProcessName, UserName	class=Endpoint Activity; device.uid; activity_name; process.name; actor.user.name.
Detection finding	SIEM/EDR: alert_name, rule_id, severity, entity, disposition	class=Detection Finding; finding.title; rule.uid; severity; resource/entity; status/disposition.

2. Why semantic layer is needed

OCSF standardizes the shape of security data, but it does not fully define your enterprise meaning. The semantic layer defines concepts, metrics, relationships, ownership rules, and policy-aware interpretations that users, dashboards, and agents can reuse.

Question users ask	What semantic layer resolves	Likely physical data needed
Show risky applications.	Application, risk score, criticality, ownership, active exposure, SLA breach.	CMDB apps, asset-app mapping, vuln findings, exposure, IAM, control data.
Which assets are internet-facing?	Approved definition of internet-facing.	Cloud public IPs, WAF/LB exposure, firewall paths, security groups, scanner evidence.
Who owns this vulnerable asset?	Authoritative owner rule.	CMDB owner first, then app mapping, then cloud tag fallback, then exception queue.
Which critical vulns violate SLA?	Critical vuln and SLA breach definitions.	Vulnerability findings, severity, first_seen, asset criticality, SLA policy table.
What is crown-jewel exposure?	Crown-jewel application definition and exposure relationship.	App criticality registry, asset mappings, internet exposure, IAM/admin paths.

Concrete semantic layer definitions

Semantic definition	Example rule
Active Asset	Asset seen in last 30 days OR present in CMDB as active OR active cloud resource. Exclude retired/decommissioned assets.
Internet-Facing Asset	Asset has public IP, public load balancer, exposed DNS, WAF path, or firewall policy that permits inbound internet traffic.
Critical Vulnerability	Normalized severity is Critical OR CVSS >= 9.0 OR exploitability flag is true and business policy classifies it as critical.
SLA Breach	Finding age exceeds SLA threshold based on severity and asset criticality. Example: Critical on crown-jewel asset > 7 days.
Application Risk Score	Weighted score using critical vuln count, exposure, exploit availability, privileged access, data sensitivity, and open SLA breaches.
Authoritative Owner	Use CMDB application owner for app assets; use cloud account owner for unmanaged cloud resources; create stewardship exception if conflict.

3. Why central catalogue is needed

A central catalogue should not become a copy of all raw cyber data. Its purpose is to catalogue data products, schemas, semantic definitions, owners, lineage, quality, certification, policy tags, and access rules. Domain projects can still own and store their data, while the catalogue provides enterprise discovery and governance.

Catalogue object	Concrete metadata captured	Why it matters
Data product	Name, description, owner, location, domain, SLA, certification status.	Users and agents know which table/API is trusted.
Schema	Columns, types, descriptions, sample values, sensitivity tags.	Avoids wrong field usage and protects restricted fields.
OCSF mapping	Source field -> OCSF field, mapping version, mapping quality.	Shows how raw scanner/EDR fields became normalized records.
Semantic concept	Definition, owner, version, approved calculation, examples.	Keeps terms like active asset, exposure, risk, SLA breach consistent.
Lineage	Raw -> OCSF -> gold entity -> dashboard/agent.	Auditability and impact analysis when source changes.
Quality	Freshness, completeness, duplicates, unmapped records, validation failures.	Consumers can trust or reject a dataset based on evidence.

Policy	Tenant scope, row/column access, PII/security tags, approved use cases.	Supports safe self-service and agent access control.
--------	---	--

Concrete catalogue entries

Entry	Example catalogue record
vuln_findings_gold	Certified BigQuery table. Owner: Vulnerability Mgmt. Refresh: hourly. Source lineage: Tenable/Qualys/Wiz raw -> OCSF vulnerability finding -> gold. Quality: 98.7% mapped resources.
asset_inventory_gold	Certified BigQuery table. Owner: Asset Platform. Refresh: daily + streaming for cloud. Authoritative for asset_id, hostname, cloud_resource_id, status, app_id.
application_risk_daily	Curated metric table. Owner: Cyber Risk. Refresh: daily. Uses semantic metric version risk_score_v3. Used by exec dashboard and agent.
Internet-Facing Asset concept	Semantic definition. Owner: Cloud Security + Network Security. Version: 2.1. Approved sources: cloud exposure, firewall path, external scanner evidence.
Asset has Vulnerability relationship	Graph edge definition. Source: vuln_findings_gold.resource_uid -> asset_inventory_gold.asset_id. Quality rule: only active assets and open findings.

4. With and without - combined decision view

Architecture	What users experience	Concrete impact
No OCSF, no semantic layer, no catalogue	Users search raw tables and ask SMEs for joins.	Every app-risk query is custom SQL. New source onboarding breaks dashboards. Agents hallucinate joins.
OCSF only	Security events/findings are consistent, but business questions still need custom joins.	Good for normalized vulnerability findings, but "risky application" still needs app ownership, exposure, SLA, and criticality logic.
Semantic layer only	Business concepts exist, but raw source mapping remains heavy.	Possible to define risk, but each scanner and EDR needs custom mapping logic in semantic definitions.
Catalogue only	Data is discoverable, but meaning and record shape may still be inconsistent.	Analysts can find tables, but still debate field meaning and write source-specific queries.
OCSF + semantic layer	Users ask in cyber concepts over normalized records.	Reliable questions such as "critical vulns on internet-facing crown-jewel apps" become reusable.
OCSF + semantic layer + central catalogue	Users and agents discover trusted data, understand meaning, and execute governed queries.	Best model for enterprise self-service, agentic workflows, auditability, and cross-domain analytics.

5. Concrete end-to-end examples

Example A - Vulnerability management

Layer	Concrete role
Raw sources	Tenable, Qualys, Wiz, container scanner, cloud security posture tool.
OCSF	Normalize each scanner output into vulnerability finding records with common severity, status, resource, CVE, first_seen, last_seen.
Semantic layer	Define active vulnerability, critical vulnerability, SLA breach, asset criticality, exploitable vulnerability, and app-level risk.
Central catalogue	Expose certified vuln_findings_gold, mapping quality, freshness, owner, lineage, and approved risk metrics.
Without all three	Teams argue whether scanner severity or CVSS is authoritative; dashboards count duplicates; app owner reports differ by team.
With all three	A dashboard and NLQ agent can answer: "Top apps with critical exploitable SLA-breached vulns" consistently.

Example B - Identity access risk

Layer	Concrete role
Raw sources	Azure AD, Okta, PAM, IAM roles, AD groups, cloud IAM policies.
OCSF	Normalize authentication/activity records and identity-related events where applicable.
Semantic layer	Define privileged identity, dormant admin, toxic combination, identity-to-asset access, break-glass account.
Central catalogue	Expose certified identity graph/data products, owner, lineage, freshness, and policy restrictions.
Without all three	Privileged means different things across AD, cloud IAM, and PAM; access-risk queries are one-off.
With all three	Agent can answer: "Which privileged identities can access internet-facing assets with critical vulns?" using approved definitions.

Example C - Cloud exposure

Layer	Concrete role
Raw sources	AWS/Azure/GCP inventory, security groups, firewall rules, load balancers, DNS, external scanner.
OCSF	Normalize cloud activity/security findings and resource-related findings where applicable.
Semantic layer	Define internet-facing, public endpoint, exposed admin port, exposed sensitive service, approved exception.
Central catalogue	Expose cloud_exposure_gold and internet-facing semantic definition with ownership and quality checks.
Without all three	One team uses public IP only; another uses load balancer exposure; another uses scanner evidence. Reports conflict.
With all three	Exposure has one governed definition that can use multiple evidence sources and show confidence/lineage.

Example D - Application risk reporting

Layer	Concrete role
Raw sources	CMDB, asset inventory, vulnerability findings, exposure, IAM, EDR alerts, backup/control data.
OCSF	Normalize security findings and events feeding the risk calculation.
Semantic layer	Define application, crown-jewel, app owner, app assets, risk score, risk drivers, exception handling.
Central catalogue	Expose application_risk_daily, metric definition version, dashboard lineage, data owner, and quality score.
Without all three	Executives see different app-risk counts from different teams. Root cause is unclear.
With all three	Executives see a certified metric with traceable inputs, owner, definition, and lineage.

Example E - Threat detection and investigation

Layer	Concrete role
Raw sources	SIEM alerts, EDR telemetry, identity logs, network logs, cloud audit logs.
OCSF	Normalize authentication, endpoint activity, network activity, and detection findings.
Semantic layer	Define suspicious lateral movement, privileged escalation, affected application, business criticality, investigation priority.
Central catalogue	Show which normalized event tables are certified, their retention, owner, freshness, and access policy.
Without all three	Investigators manually correlate tool-specific logs and may miss app/business context.
With all three	Investigation agent can correlate event evidence with asset/app/identity context and explain why it is high priority.

6. Central catalogue in a multi-domain GCP model

In a separated domain-project model, the central project does not need to own all cyber data. It can own the catalogue, semantic registry, OCSF mapping registry, and pointers to domain-owned BigQuery tables, graph services, APIs, and dashboards.

Domain GCP Projects	
Cyber Asset Project	-> asset_inventory_gold
Vulnerability Project	-> vuln_findings_gold
IAM / AD Project	-> identity_entitlements_gold
Cloud Security Project	-> cloud_exposure_gold
PKI Project	-> certificate_inventory_gold
Threat Project	-> detection_findings_gold
Central Catalogue Project	
Data product registry	
Semantic entity/metric registry	
OCSF mapping registry	
Ownership + stewardship metadata	
Lineage + quality + certification	
Policy tags + access rules	
Pointers to BigQuery, graph, APIs, dashboards	
Consumers	
Dashboards, NLQ/NL2SQL, agents, governance reports, data discovery portals	

Concrete central catalogue purpose

Purpose	Concrete answer it provides
Discovery	Which dataset should I use for assets, vulnerabilities, identities, exposures, and app risk?
Trust	Is this dataset certified? When was it refreshed? What quality checks passed or failed?
Ownership	Who owns this data product and who approves definition changes?
Lineage	Which raw sources feed this metric, and which dashboards/agents consume it?
Policy	Can this user/agent query this tenant, row, column, or sensitive field?
Semantic grounding	What does "internet-facing" or "SLA breach" mean and which version is approved?
Onboarding	When a new source arrives, where do mappings, quality checks, owners, and certified outputs get registered?

7. Concrete query translation without optimization details

Even without focusing on performance optimization, the combination helps translate a user question into trusted data access. The semantic layer and catalogue reduce guessing.

User question	Catalogue helps find	Semantic layer resolves	OCSF contributes
Show critical vulns on internet-facing payment apps.	Certified app, asset, vuln, exposure data products.	Payment app, critical vuln, internet-facing, active finding.	Normalized vulnerability finding severity/status/resource.
Which privileged identities can access exposed assets?	Certified identity entitlement and exposure datasets.	Privileged identity, access relationship, exposed asset.	Normalized identity/auth activity where used.
Which apps have expired certificates?	Certified certificate inventory and app mapping.	Expired certificate, app ownership, app criticality.	Security findings/events related to certificates if normalized.
Show ownerless crown-jewel assets.	Asset inventory, CMDB, crown-jewel registry.	Ownerless, crown-jewel, active asset, authoritative owner.	Normalized resource identifiers support correlation.

8. Practical takeaway

Need	Use
Normalize cyber data from many tools.	OCSF.
Create common enterprise meaning for cyber concepts.	Semantic layer.
Find trusted datasets, owners, lineage, quality, and policies.	Central catalogue.
Support NLQ/NL2SQL/agents safely.	Semantic layer + central catalogue + policy metadata.

Support domain-owned GCP projects without copying all data centrally.	Central catalogue stores metadata and pointers; domain projects store data.
Produce consistent executive cyber risk reporting.	OCSF for normalized inputs, semantic layer for metrics, catalogue for certification and lineage.

Bottom line: OCSF makes cyber records consistent. The semantic layer makes the records meaningful in business/cyber terms. The central catalogue makes those data products and definitions discoverable, trusted, governed, and reusable across domains.